

原版技术性开发系列教程

第 1 册

命令系统 Ciallo!

徐木弦 主编
(急招：编写组成员)

第一版序言

Minecraft 拥有多种多样的玩法，生存、PVP、PVE、模组、建筑、红石电路这些为众多玩家所熟知的玩法自成体系，玩家可以自由选择其中的某一方面深入研究。在这些玩法中，比较默默无闻的一种玩法可能便是广义上的命令，即包含了 MC-CMD（命令）、资源包和数据包的系统。

在没有系统地接触命令系统前，玩家对于这个系统不了解、甚至完全没有意识到这个系统的存在都是有可能的。对于一些仅对命令系统做出更新的快照版本，一些玩家可能认为它们是“无用的”，即使这些更新在大部分情况下丰富了命令系统，使得服务器的运营、冒险地图及原版模组的制作更方便。在较为早期的版本中，命令更多依靠命令方块运行，这使得一些玩家错误地认为冒险地图或原版模组中的机关是由纯粹的红石电路运行的，并将这些机关的巧妙之处完全归结于红石电路的运作而完全忽略命令的作用。

这是因为命令系统更多地呈现一种“在后台运行”的状态，玩家无法在明面上查看它们的运行逻辑，更多地只是感受它们带来的效果。其次，命令系统是一个“入门难、精通容易”的板块，学习的门槛较高，一些基本的概念，如权限等级、命名空间 ID、加载等级、坐标、目标选择器、JSON、NBT、记分板等，都是难于理解而对命令学习至关重要的知识点。再者，普通玩家在接触到命令系统时，更多地是熟悉诸如 `/kill`、`/gamemode` 这类带有简单参数的命令写法，而不会深究命令中各参数的意义，浅尝辄止的行为让玩家不大可能形成命令系统是一个完备体系的认识。最后，玩家在游戏时一般不会启用命令，而大部分命令所需的权限等级较高（即需要启用命令才能使用），因此一般玩家很少会与命令产生接触。

不过，命令系统应用范围极广，它对游戏的作用也是显而易见的，服务器的运行、冒险地图的制作、原版模组的制作均需要使用命令系统。对于游戏原有内容的修改和新内容的制作，如游戏外观的修改、粒子动画效果、自定义战利品、自定义物品等，均可以由命令系统来完成。命令系统提供了一个在不修改源代码情况下的开发环境，使得 Minecraft 几乎成为了一款“无限”的游戏，唯一有限的也许便是玩家的想象力。同时，命令系统也极大地丰富了游戏社区、论坛。

在历代版本更新中，命令系统也随之有过大大小小的改善。在早期的版本中，命令一般只能借由聊天栏和命令方块执行，而命令方块是一种红石元件，因此彼时命令的自动化执行对于红石电路的依赖性较强，只能将红石信号转化为命令执行的信号或条件。然而在后续的版本中，函数及数据包的问世使得命令能够脱离红石电路单独运行，并随着数据包的不断完善，命令系统的研

究和应用则更加方便。一些冒险地图真正做到了零命令方块化,命令的执行全部转交给数据包完成。因此命令方块及红石电路的教程则放在了附录中,不使用数据包仅使用命令方块时可以参阅这些内容。

对于这样一个较为完备的体系,笔者在前人既有教程的基础上,理清思路,加以整理,为之编写完整的命令教程。在内容安排上,优先讲述基本概念,如权限等级、命令执行上下文、命名空间 ID、数据包标签、参数、区块及加载等级等,这些概念是能够执行命令的最基本条件。在绪论之后是命令系统的五大重要基础板块:坐标、目标选择器、JSON、NBT 和记分板,这些内容是需要熟练掌握的基本功。在第七章,我们主要学习命令 `/execute`,这条命令能够修改命令执行上下文,一般可以认为是整个命令系统中最强大的命令。一些零散的命令则放到了第八章中讲解。资源包和数据包则被安排到了其他教程中,这些内容是命令系统的进阶部分,一定程度上脱离了狭义的命令体系,却也是命令系统的精华部分。学有余力的读者可进行《资源包》和《数据包》两本教程的学习。

在系统学习命令系统前,读者需要了解到的一点是,其他玩法领域可能更多地追求稳定的游戏环境,因此可能会选择旧的游戏版本。而命令系统随着版本更新不断完善,使得命令玩家不自觉地追求新版本,因此有关命令的讨论一般都基于最新版游戏。一些问题,诸如“在 1.8 版本中这个命令怎么写”,则可能得不到答复,因为命令系统在扁平化时经历了巨大的改动,使得改动前后几乎是两个完全不同的系统。同时,不同平台的命令系统也是不互通的,本教程仅适用于 Java 版,部分内容与基岩版不同,因此无法作为基岩版命令教学的参考。

那么在学习命令之前需要掌握哪些预备知识呢?首先需要对原版的 Minecraft 有足够的了解,因为命令基本上涉及了游戏中的方方面面。其次可以适当学习一些英语,命令的参数中有大量的英语单词,如果熟悉它们的意思将会极大提升命令编写的效率。本教程在论述的过程中,会经常使用高等数学、线性代数和概率论与数理统计的语言,读者可先行了解与命令系统紧密相关的一些数学知识,如向量代数与空间解析几何、矩阵与变换、随机事件及概率分布等。计算机图形学的一些内容也包括在内。此外,大家还可以提前学习一些编程思想,建立严密的逻辑思维能力,懂得如何指挥电脑(游戏)进行工作。没有相关编程基础并不意味着完全不能学好命令,编程思维也可以在命令学习的过程中逐步建立。

本教程仅作为基础教程使用,即仅介绍命令系统中所有的基本概念,并在此基础上进行一定的应用、扩展,其中应用层面主要面向冒险地图和原版模组的制作。一些较为高级的算法则不在本教程范围之内。其中带“*”号的是选择性学习内容,它们所述的一些是不常用的内容,一些则是《资源包》或《数据包》两本教程的相关内容,想进修《资源包》和《数据包》的读者需留意这些带“*”号知识点。

限于笔者的知识水平,本教程必有一定疏漏之处,望广大读者斧正!

编者

2024 年 1 月 24 日

第二版序言

自 2023 年发布的 1.19.4 起，Minecraft 的技术性开发板块进入了又一个新时代，这可谓是自 2012 年骇人更新加入命令方块、2018 年水域更新扁平化以来第三次革命性的巨变。这次“巨变”的主要体现在于：逐步开放了很多 API，即将很多固有注册表转为可写注册表，使得原先需要大量命令模拟的效果用数据包自定义即可，这无疑大大省去了技术性开发的成本。从这两年 Minecraft Java 版的更新迭代可以看出，Minecraft 的开发重点逐渐地由加入新的实质性游戏内容转为对数据包、资源包的支持。新游戏内容更新的占比下降，技术性更新的占比上升。数据包、资源包的版本号更迭已由先前的一个大版本一次变为一个快照一次，迭代速度越来越快。以数据包为例，2022 年数据包的版本号为 10，彼时距数据包的加入仅过了四年多；而截止至 2024 年底，数据包的版本号已达到了 61。

许多玩家对自 1.17 以来的 Minecraft 开发极不满意，有些人认为 Minecraft 近几年并没有革命性的更新出现，每个版本的更新内容也极少，对游戏深度的挖掘不够，“敷衍了事”。对于每个快照更新介绍中大篇幅的技术性更新内容，大量玩家则视若无睹。这个现象直接反映了普通玩家对技术性开发这一板块认识的欠缺，正如第一版序言所说，“玩家对于这个系统不了解、甚至完全没有意识到这个系统的存在”。为此，对玩家进行技术性开发板块知识的普及很有必要。

不得不承认的是，技术性开发确实是一种门槛较高的玩法，其学习成本较高，容错率低，开发周期长，且开发过程不可见，因此难于在社区中引起较大的讨论度。目前，各大社区平台仍然缺乏专门的技术性开发论坛，现有的教程也零碎且不完整。社区的讨论主要集中于原版游戏内容、Mods 玩法等，鲜有技术性开发有关的资料，这使其搜索难度较大。技术性开发玩家通常“单打独斗”，缺少交流，进一步限制了有开发想法的玩家学习进步。为此，为已入门的技术性开发玩家编写系统性的、完整的教程很有必要。

原版技术性开发的命令、资源包、数据包三个板块在本系列教程中分作三部分别讲述。对于其中的一些基本概念，如注册表、坐标、目标选择器、文本组件、NBT、记分板、属性、存档格式等，为了便于教程编纂，将其放在《命令系统》一书中与命令语法一起说明。命令是数据包的一个子集，用于函数中，数据包为它提供了程序化运行的环境。在早先的版本中，命令通常是红石电路的一个子集，用于命令方块中，由红石信号控制命令的执行。虽然现在主流的开发环境是数据包，但仍有玩家使用命令方块红石电路进行开发，因此本系列教程中仍保留对命令方块红石电

路的讲解,《命令系统》书中的一些例子则提供了使用数据包和使用命令方块红石电路的两种解法,但侧重于使用数据包,而在《数据包》中则仅提供使用数据包的解法。资源包是相对较独立的一个板块,其内容中仅少部分与数据包有关联,对美术设计的要求较高,部分开发资源包的玩家专注于资源包,不会去开发数据包。然而在开发实践中数据包与资源包的联系却越来越紧密,仅依靠数据包能够实现的效果有限,与资源包配合开发能得到更好的效果。

这一版的《命令系统》教程相较上一版而言,对调了一些章节使得教程整体的逻辑更合理。同时新增了一些章节,新增的内容大多拆分自原本的章节,但对这些被拆分的内容有了更详尽的叙述。原本教程中零散的内容在这一版中均尽可能地归到一类下。经过这些变动后,教程的整体章节较先前的版本有了很大的不同。产生这些变动的主要原因是文本组件的存储格式由 JSON 变为了 SNBT,即使在数据包中文本组件仍使用 JSON 格式。从教程逻辑上讲,文本组件应合并至“NBT 格式”一章,但上一版的“NBT 格式”一章已有 11 个小节,内容量极其庞大,若再塞入文本组件的全部内容,只会使“NBT 格式”一章显得更臃肿。况且属性、状态效果这些内容依照存储格式也归到了实体格式或物品堆叠组件格式之下,将这些归类的内容全部放入“NBT 格式”一章更不适宜。于是,原本经典的“绪论——坐标——目标选择器——原始 JSON 文本——NBT 格式——记分板——命令 `/execute`——杂项命令”的教程章节顺序已不适用。经过考虑,这一版的教程章节变动安排如下:

1. “绪论”、“坐标与区块”、“UUID 与目标选择器”三章的顺序不变,仅调整这三章内部小节的顺序及编排。
2. 删去原本的“NBT 格式”一章,并将其重命名为“NBT 与 JSON 格式”,新的章节保留原本“NBT 格式”一章的概述、NBT 路径、命令 `/data` 的语法。同时将原本“原始 JSON 文本”一章的“JS 对象表示法”一节移入新的“NBT 与 JSON 格式”。随后将概述一节中涉及 SNBT 和 JSON 相互转换的内容移出,单独设置一个小节。
3. 将新的“NBT 与 JSON 格式”提前至第四章,将“原始 JSON 文本”改名为“文本组件”并后移至第五章。“文本组件”一章将同时讲解其 SNBT 和 JSON 形式。
4. 对于从原本“NBT 格式”一章中移出的其他内容,结合属性、状态效果,单独开设新的一章“存档格式”作为第六章,同时合并原本附录 II “游戏文件”的内容至这一章的概述小节。新的“存档格式”包含命令存储格式、区域文件格式、方块实体格式、实体格式、属性格式、状态效果格式、标记格式、展示实体格式、交互实体格式、玩家格式和物品堆叠组件格式。
5. “记分板”、“命令的执行”、“其他分类的命令”分别顺延至第七、八、九章。“命令的执行”更名为“命令 `/execute`”。
6. 由于章节合并,“其他分类的命令”中“状态效果”、“属性”两节被删除。
7. 删除原本的附录 II “游戏文件”。

受限于编者的知识水平,本教程一定有不足之处,望广大读者斧正!

编者

2025 年 2 月 7 日

目 录

第一章 绪论	1
1.1 注册表与数据值	2
1.1.1 注册表 *	2
1.1.2 扁平化 *	8
1.1.3 命名空间 ID	9
1.1.4 数据包标签	12
1.2 命令	12
1.2.1 命令参数	12
1.2.2 命令的输入	14
1.2.3 权限等级与限制条件	16
1.2.4 命令的解析 *	18
1.2.5 命令上下文	19
1.3 JSON 格式	20
1.3.1 JSON 数据类型	21
1.3.2 JSON 的转义序列	23
1.3.3 多级转义 *	25
1.4 文件结构	25
1.5 数据包	25
1.6 资源包	26
1.7 游戏机制	26
1.7.1 端	27
1.8 服务器	28
1.9 带有简单参数的命令指引	28
第二章 坐标	29
2.1 坐标系与坐标	30
2.1.1 坐标的命令参数类型	30
第三章 区块格式	33

3.1 技术性实体	33
-----------------	----

第一章 绪论

原版技术性开发, **Minecraft Wiki** 称为“Java 版可自定义内容”, 是由命令、资源包、数据包及相关的组件附件组合成的一个板块。技术性开发成果丰富, 这些成果即是社区玩家常用的 **Mods**、冒险地图、数据包、资源包、服务器等。**Minecraft** 的技术性开发大致分为 **Mods** 开发和原版开发, 其区别在于是否对游戏的源代码进行了修改。

本系列教程针对的是原版技术性开发, 这一部分玩家的工作方向通常为制作冒险地图、开发原版模组、制作资源包或者管理服务器。

1.1 注册表与数据值

Minecraft 有许多不同的游戏资源，如草方块、石头、箭、铁锹、猪等，对这些游戏资源进行分类，可以将草方块、石头分为方块，箭、铁锹分为物品，猪划分至实体。方块、物品、实体显然是区分这些游戏资源的“大类”。**注册表 (Registry)** 就是对不同资源进行分类管理的机制。除分类管理外，还需要给予每种资源一个独特的“身份证”，目的是与别的资源区分开来，唯一地映射到注册表内的给定值。这些“身份证”被称为游戏资源的数据值，或称 ID。

1.1.1 注册表 *

注册表可分为以下两类：**固有注册表 (Built-in registry)** 和 **可写注册表 (Writable registry)**。无论资源位于什么类型的注册表，**游戏都只会识别已被注册的资源**。

下面列举了 Minecraft 中所有的资源类型：

一、固有注册表

固有注册表存储硬编码的游戏内容，除非修改源代码，否则其中的内容不可更改。

表 1.1 固有注册表

注册表	资源类型	数据包路径	默认值
ACTIVITY	生物 AI	activity	
ATTRIBUTE	属性	attribute	
ATTRIBUTE_TYPE	环境属性类型	attribute_type	
BIOME_SOURCE	生物群系源	worldgen/biome_source	
BLOCK	方块	block	air
BLOCK_ENTITY_TYPE	方块实体类型	block_entity_type	furnace
BLOCK_PREDICATE_TYPE	方块谓词类型	block_predicate_type	
BLOCK_STATE_PROVIDER_TYPE	方块状态提供者类型	worldgen/block_state_provider_type	
BLOCK_TYPE	方块类型	block_type	
CARVER	雕刻器类型	worldgen/carver	
CHUNK_GENERATOR	区块生成器类型	worldgen/chunk_generator	
CHUNK_STATUS	区块状态	chunk_state	empty
COMMAND_ARGUMENT_TYPE	命令参数类型	command_argument_type	
CONSUME_EFFECT_TYPE	消耗使用效果类型	consume_effect_type	
CREATIVE_MODE_TAB	创造标签页	creative_mode_tab	

(续表)

注册表	资源类型	数据包路径	默认值
CUSTOM_STAT	统计信息	custom_stat	
DATA_COMPONENT_PREDICATE_TYPE	数据组件谓词类型	data_component_predicate_type	
DATA_COMPONENT_TYPE	数据组件类型	data_component_type	
DEBUG_SUBSCRIPTION	调试订阅	debug_description	
DECORATED_POT_PATTERN	饰纹陶罐图案类型	decorated_pot_pattern	
DENSITY_FUNCTION_TYPE	密度函数类型	worldgen/density_function_type	
DIALOG_ACTION_TYPE	对话框操作类型	dialog_action_type	
DIALOG_BODY_TYPE	对话框主体类型	dialog_body_type	
DIALOG_TYPE	对话框类型	dialog_type	
ENCHANTMENT_EFFECT_COMPONENT_TYPE	魔咒效果组件类型	enchantment_effect_component_type	
ENCHANTMENT_ENTITY_EFFECT_TYPE	魔咒实体效果类型	enchantment_entity_effect_type	
ENCHANTMENT_LEVEL_BASED_VALUE_TYPE	魔咒等级依赖函数类型	enchantment_level_based_value_type	
ENCHANTMENT_LOCATION_BASED_EFFECT_TYPE	魔咒位置依赖函数类型	enchantment_location_based_effect_type	
ENCHANTMENT_PROVIDER_TYPE	魔咒提供器类型	enchantment_provider_type	
ENCHANTMENT_VALUE_EFFECT_TYPE	魔咒值效果类型	enchantment_value_effect_type	
ENVIRONMENT_ATTRIBUTE	环境属性	environment_attribute	
ENTITY_SUB_PREDICATE_TYPE	实体子谓词类型	entity_sub_predicate_type	
ENTITY_TYPE	实体类型	entity_type	pig
FEATURE	地物类型	worldgen/feature	
FEATURE_SIZE_TYPE	树木生成的最小空间要求类型	worldgen/feature_size_type	
FLOAT_PROVIDER_TYPE	浮点数提供器类型	float_provider_type	
FLUID	流体类型	fluid	empty
FOLIAGE_PLACER_TYPE	树叶放置器类型	worldgen/foilage_placer_type	
GAME_EVENT	游戏事件	game_event	step
GAME_RULE	游戏规则	game_rule	step
HEIGHT_PROVIDER_TYPE	高度提供器类型	height_provider_type	
INCOMING_RPC_METHOD	服务端管理协议中的请求方法	incoming_rpc_methods	
INPUT_CONTROL_TYPE	对话框输入控件类型	input_control_types	

(续表)

注册表	资源类型	数据包路径	默认值
INT_PROVIDER_TYPE	整数提供器类型	int_provider_type	
ITEM	物品	item	air
SLOT_SOURCE_TYPE	槽位源类型	slot_source_type	
LOOT_CONDITION_TYPE	战利品表谓词类型	loot_condition_type	
LOOT_FUNCTION_TYPE	物品修饰器类型	loot_function_type	
LOOT_NBT_PROVIDER_TYPE	战利品表相关 NBT 源类型	loot_nbt_provider_type	
LOOT_NUMBER_PROVIDER_TYPE	值提供器类型	loot_number_provider_type	
LOOT_POOL_ENTRY_TYPE	抽取项类型	loot_pool_entry_type	
LOOT_SCORE_PROVIDER_TYPE	分数提供器类型	loot_score_provider_type	
MAP_DECORATION_TYPE	地图图标类型	map_decoration_type	
MATERIAL_CONDITION	地表规则条件	worldgen/material_condition	
MATERIAL_RULE	地表规则	worldgen/material_rule	
MEMORY_MODULE_TYPE	生物记忆	memory_module_type	dummy
MENU	屏幕类型	menu	
MOB_EFFECT	状态效果	mob_effect	
NUMBER_FORMAT_TYPE	记分板分数显示样式类型	number_format_type	
OUTGOING_RPC_METHOD	服务端管理协议中的通知方法	outgoing_rpc_methods	
PARTICLE_TYPE	粒子类型	particle_type	
PLACEMENT_MODIFIER_TYPE	放置修饰器类型	worldgen/placement_modifier_type	
PERMISSION_CHECK_TYPE	命令权限检查类型	permission_check_type	
PERMISSION_TYPE	命令权限类型	permission_type	
POINT_OF_INTEREST_TYPE	兴趣点类型	point_of_interest_type	
POOL_ALIAS_BINDING	模板池映射	worldgen/pool_alias_binding	
POSITION_SOURCE_TYPE	位置源类型	position_source_type	
POS_RULE_TEST	位置规则测试类型	pos_rule_test	
POTION	药水效果	potion	
RECIPE_BOOK_CATEGORY	配方书分类标签	recipe_book_category	
RECIPE_DISPLAY	预览配方类型	recipe_display	
RECIPE_SERIALIZER	配方序列化/反序列化器	recipe_serializer	

(续表)

注册表	资源类型	数据包路径	默认值
RECIPE_TYPE	配方类型	recipe_type	
REGISTRY	注册表	root	
ROOT_PLACER_TYPE	树根放置器类型	worldgen/root_placer_type	
RULE_TEST	规则测试	rule_test	
RULE_BLOCK_ENTITY_MODIFIER	方块实体数据修饰器	rule_block_entity_modifier	
SENSOR_TYPE	感受器类型	sensor_type	
SLOT_DISPLAY	预览槽位类型	slot_display	
SOUND_EVENT	声音事件	sound_event	
SPAWN_CONDITION_TYPE	通用变种选择器类型	spawn_condition_type	
STAT_TYPE	统计类型	stat_type	
STRUCTURE_PIECE	结构片段	worldgen/structure_piece	
STRUCTURE_PLACEMENT	结构放置方式	worldgen/structure_placement	
STRUCTURE_POOL_ELEMENT	结构模板池元素	worldgen/structure_pool_element	
STRUCTURE_PROCESSOR	结构处理器	worldgen/structure_processor	
STRUCTURE_TYPE	结构类型	worldgen/structure_type	
TEST_ENVIRONMENT_DEFINITION_TYPE	测试环境类型	test_environment_definition_type	
TEST_FUNCTION	硬编码测试函数	test_function	
TEST_INSTANCE_TYPE	测试实例类型	test_instance_type	
TICKET_TYPE	加载标签及计算标签类型	ticket_type	
TREE_DECORATOR_TYPE	树额外装饰器类型	worldgen/tree_decorator_type	
TRIGGER_TYPE	触发器类型	trigger_type	
TRUNK_PLACER_TYPE	树干放置器类型	worldgen/trunk_placer_type	
VILLAGER_PROFESSION	村民职业	villager_profession	none
VILLAGER_TYPE	村民类型	villager_type	plain

二、可写注册表

可写注册表允许数据包通过数据反序列化器向其中添加自定义（或称数据驱动）的游戏内容。

表 1.2 可写注册表

注册表	资源类型	数据包路径
ADVANCEMENT	进度	advancement
BANNER_PATTERN	旗帜图案	banner_pattern
BIOME	生物群系	worldgen/biome
CAT_VARIANT	猫的变种	cat_variant
CHAT_TYPE	聊天类型	chat_type
CHICKEN_VARIANT	鸡的变种	chicken_variant
CONFIGURED_CARVER	已配置的雕刻器	worldgen/configured_carver
CONFIGURED_FEATURE	已配置的地物	worldgen/configured_feature
COW_VARIANT	牛的变种	cow_variant
DAMAGE_TYPE	伤害类型	damage_type
DENSITY_FUNCTION	密度函数	worldgen/density_function
DIALOG	对话框	dialog
DIMENSION	维度	dimension
DIMENSION_TYPE	维度类型	dimension_type
ENCHANTMENT	魔咒数据格式	enchantment
ENCHANTMENT_PROVIDER	魔咒提供器	enchantment_provider
FLAT_LEVEL_GENERATOR_PRESET	超平坦世界生成预设	worldgen/flat_level_generator_preset
FROG_VARIANT	青蛙的变种	frog_variant
INSTRUMENT	山羊角乐器	instrument
ITEM_MODIFIER	物品修饰器	item_modifier
JUKEBOX_SONG	唱片机曲目	jukebox_song
LEVEL_STEM	维度	dimension
LOOT_TABLE	战利品表	loot_table
MULTI_NOISE_BIOME_SOURCE_PARAMETER_LIST	多噪声参数列表	worldgen/multi_noise_biome_source_parameter_list
NOISE	噪声	worldgen/noise
NOISE_SETTINGS	噪声设置	worldgen/noise_settings
PAINTING_VARIANT	画的变种	painting_variant
PIG_VARIANT	猪的变种	pig_variant
PLACED_FEATURE	已放置的地物	worldgen/placed_feature
PREDICATE	谓词	predicate
PROCESSOR_LIST	处理器列表	worldgen/processor_list
RECIPE	配方	recipe

(续表)

注册表	资源类型	数据包路径
STRUCTURE	已配置的结构地物	worldgen/structure
STRUCTURE_SET	结构集	worldgen/structure_set
TEMPLATE_POOL	结构池	worldgen/template_pool
TEST_ENVIRONMENT	测试环境	test_environment
TEST_INSTANCE	测试实例	test_instance
TIMELINE	时间线	timeline
TRIAL_SPAWNER_CONFIG	试炼刷怪笼配置	trial_spawner
TRIM_MATERIAL	盔甲纹饰材料	trim_material
TRIM_PATTERN	盔甲纹饰图案	trim_pattern
WOLF_VARIANT	狼的变种	wolf_variant
WORLD_PRESET	世界预设	worldgen/world_preset
ZOMBIE_NAUTILUS_VARIANT	僵尸鹦鹉螺变种	zombie_nautilus_variant

三、不属于任何注册表的游戏资源

有一些游戏资源不属于任何注册表,这些资源包括数据包内的函数、结构模板以及资源包内的所有内容。这些资源类型中部分都与可写注册表的性质类似,即可以自定义写入资源;部分则不能增添新的资源,但可以修改已有资源的配置文件。

1. 数据包内容

- (1) 函数: 可写, 即 `function` 路径下的内容。
- (2) 结构模板: 可写, 即 `structure` 路径下的内容。

2. 资源包内容

- (1) 纹理图集: 位于资源包内路径 `atlases`。
- (2) 方块状态: 位于 `blockstates`。
- (3) 纹饰图案: 位于 `equipment`。
- (4) 字体: 可写, 位于 `font`。
- (5) 物品模型映射: 位于 `items`。
- (6) 模型: 包括方块模型和物品模型, 位于 `models`。
- (7) 粒子: 位于 `particles`。
- (8) 着色器: 可写, 位于 `shaders`。
- (9) 后处理管线: 位于 `post_effect`。
- (10) 声音: 可写, 位于 `sounds`。
- (11) 纹理: 可写, 位于 `textures`。
3. 属性修饰符: 可写, 存储于所属物品的堆叠组件内。
4. Boss 栏: 可写, 存储于存档文件夹中的 `level.dat`。
5. 命令存储: 可写, 存储于存档文件夹中的 `data\command_storage_minecraft.dat`。
6. 随机序列: 可写, 存储于存档文件夹中各自维度的 `data\random_sequences.dat` 文件内。

1.1.2 扁平化 *

Minecraft 的历次版本更新都会对某一些特定的系统进行优化和更改,比如:战斗更新对 PVP 机制进行了颠覆性的更改,使得 1.9 之前和之后的 PVP 是两个完全不同的系统。命令系统也经历过类似的大幅度更改,这便是随着水域更新进行的**扁平化 (The flattening)**。

在 Minecraft 开发之初,由于游戏资源的数量有限,只需要使用 1 字节就可以设置所有游戏资源的 ID。在历次版本更新中, Minecraft 的方块、物品数量越来越多,特别是自缤纷更新以来,方块的数量呈爆炸式增长。在扁平化之前,为了应对这些不断增多的游戏资源,一种解决办法是将一大类全部收归到某一个特定的 ID 中,用这个 ID 来表示这一类方块,然后在后面附加一个 Damage 值来表示这一类方块中的某一种。比如花岗岩属于石头一类,石头的数字 ID 为 1,而花岗岩的 Damage 值为 1,所以在旧版本中给予玩家一块花岗岩的命令为:

```
give @p 1 1 1
```

1.1

可以看到这条命令中有三个参数,第一个 1 为石头的 ID,第二个 1 为数量,第三个 1 为 Damage 值。这种表示方式的底层逻辑是:在访问花岗岩时,必须先访问上一级的数字 ID,再访问 Damage 值,如此才能映射至花岗岩这个值。

缤纷更新做了一个小修改:即启用了部分英文 ID,于是在 1.8 中给予玩家一块花岗岩的命令变为:

```
give @p stone 1 1
```

1.2

但是缤纷更新做出的这种更改是不完全的,虽然在命令的主体部分将数字 ID 替换成了英文 ID,但是方块的 Damage 值仍然存在,这种一级 ID——Damage 值的映射方式没有改变。

时间来到了 2017 年,水域更新加入了大量的游戏内容,原先的映射方式已不能适应新版本。于是,Java 版 1.13 的更新基本上删除了所有的数字 ID,使得每一个游戏资源都有其独立的英文 ID。同时也删除了用 Damage 值映射方块的办法,去除了中间层,使得资源的映射方式只需要一个键名即可,这一过程便被称作“扁平化”^①。比如,在 1.13 中给予玩家一块花岗岩的命令为:

```
give @p granite 1
```

1.3

其中参数 granite 为花岗岩的键名,1 为物品数量。扁平化对所有需要 ID 的对象都进行了修改,包括但不限于方块、物品、实体、生物群系、粒子、声音事件和画。其具体内容可分为以下几类:

1. 拆分

拆分是最能体现扁平化过程的一类。此举移除了用于指定大类下某一种方块的 Damage 值,使得一类方块下的每一种方块都有其独立的 ID。举例:stone 一类共有七种不同的方块:石头、花岗岩、磨制花岗岩、闪长岩、磨制闪长岩、安山岩和磨制安山岩,分别对应 0~6 的 Damage 值。拆分后这七种方块都被给予了独立的 ID: stone、granite、polished_granite、diorite、polished_diorite、andesite 和 polished_andesite。

2. 重命名

^① 并非所有数字 ID 都被移除,至今 Minecraft 仍保留了部分需要使用数字 ID 的地方。

顾名思义，该类即对原有的英文 ID 进行重命名。有相当一部分重命名是为了迎合方块或物品的英文名称。举例：草方块在扁平化前的 ID 为 `grass`，扁平化后被重命名为 `grass_block`。

3. 重新分类

这种操作常见于台阶和由双台阶组成的完整方块。扁平化前的台阶和双台阶有两种不一样的 ID，扁平化后取消双台阶的 ID，并对台阶的下属分类进行拆分，同时取消相关的 Damage 值。举例：双木台阶被取消，统一更换为木制台阶，同时 ID 拆分为橡木、云杉、白桦、丛林木、金合欢和深色橡木。

4. 合并

合并常见于有不同方块状态的方块。举例：燃烧的熔炉和熔炉有不同的 ID，现合并为熔炉一种，同时将是否燃烧设定为方块状态。

1.1.3 命名空间 ID

游戏资源的指定有一个前提是这些对象的 ID 相互之间不能混淆。数字 ID 及使用 Damage 值作区分的指定方法能够避免对象之间的冲突，但扁平化后这种指定方法便无效了。为此在当前的版本中统一使用（**赋**）命名空间 ID（**Namespaced identifier**）来映射注册表内的值。

命名空间 ID，又称（**赋**）命名空间标识符、资源路径（**Resource location**）、资源标识符（**Resource identifier**）或命名空间字符串（**Namespaced string**），是字符串化的映射方式。无论命名空间 ID 用于映射何种对象，它们都具有同一的表达方式：

```
<namespace>:<path>
```

1.4

其中的参数：

`<namespace>` ——命名空间。

`<path>` ——路径。

在写法上，除用于分割命名空间和路径的冒号 `:` 外，其中所有的字符都只能为合法字符。合法字符包含以下几类：

1. 数字：`0123456789`；
2. 小写字母：`abcdefghijklmnopqrstuvwxyz`；
3. 下划线：`_`；
4. 连字符：`-`；
5. 点：`.`。

小提示

大写字母在命名空间 ID 中为非法字符！

除此之外所有的字符均为非法字符，包括汉字、平假名、片假名、西里尔字母、希腊字母、制表符、几何图形符、`+` 等符号。斜杠 `/` 比较特殊，它在命名空间中为非法字符，但是在路径中可用于分割目录的不同层级，以下会有详细说明。

一、命名空间 ID 的实际意义

小提示

原版游戏中大部分对象都使用命名空间 `minecraft`，但是六种基本命令参数类型（小节 1.2.1）却使用命名空间 `brigadier`。

命名空间（Namespace） 是游戏资源的区界，它位于资源类型的父层级，所有来自 Minecraft 原版游戏的资源均位于命名空间 `minecraft`。通过不同的自定义命名空间可以将新增的内容和原版内容区分开来，以防止新内容和原版内容、新内容和其他新内容之间产生冲突。例如，有两个命名空间 ID `minecraft:something` 和 `custom:something`，它们指定的是两个不同的对象，因为它们的命名空间不同，前者为 `minecraft`，后者为 `custom`，即使两者拥有相同的路径（名称）`something`。

部分游戏资源在命名空间下有一定的文件路径，尤其是资源包、数据包内容，这时就可以使用 `/` 以表明它们的文件路径。这里命名空间实际上是一个文件夹，这个文件夹的结构一般是 `<命名空间>\<资源类型>\<文件夹1>\<文件夹2>\...<文件夹n>\<文件名>.<后缀>`。若要映射到这个文件，则命名空间 ID 的写法为

```
<命名空间>:<文件夹 1>/<文件夹 2>/...<文件名>
```

1.5

注意以下几点：

1. 资源类型不作为命名空间 ID 的一部分；
2. 资源路径的子文件夹至文件之间的一系列路径必须完整；
3. 文件名后缀不写。

下面举两个例子以说明之：

例 1.1

有数据包函数文件路径为 `minecraft\function\load.mcfunction`，试用命名空间 ID 指定之。

解

这里 `function` 为资源类型，`.mcfunction` 为文件的后缀。故命名空间 ID 为

```
minecraft:load
```

1.6

例 1.2

有资源包纹理文件路径为 `minecraft\textures\block\command_block_front.png`，试用命名空间 ID 指定之。

解

这里 `textures` 为资源类型，`.png` 是文件后缀，依照其路径将命名空间 ID 写为

```
minecraft:block/command_block_front
```

1.7

二、命名空间 ID 字符串的识别与一般写法

命名空间 ID 是形如 `<字符串>:<字符串>` 的字符串，游戏在识别、调用相关对象时，如果冒号 `:` 存在，会将冒号前的内容视作命名空间，其中不能出现斜杠 `/`；将冒号后的内容视作路径，可

以出现斜杠。为了保证识别的正确性，整个字符串中最多只能出现一个冒号，否则识别会出现错误。如果冒号不存在，字符串的形式为 `<字符串>`，则游戏会直接将整个字符串直接识别为路径，并默认命名空间为 `minecraft`，有些时候可以适当减少书写命名空间，但省略命名空间的行为仍是不被建议的：虽然命名空间在原版大部分需要 ID 的地方上不是必须的，但在一些情况下命名空间是必须的，包括但不限于 NBT 路径中指定 ID 的节点。鉴于读者在实践的过程中可能会忘记必须添加命名空间的地方，或是混淆原版内容和自定义的内容，那么最好还是完整地书写命名空间 ID。

例 1.3

下列命名空间 ID 的识别结果为何？

`something`

1.8

`minecraft:something`

1.9

`custom:something`

1.10

`minecraft/custom:something`

1.11

`minecraft/something`

1.12

`minecraft:Something`

1.13

`minecraft:custom:something`

1.14

解

以上各命名空间 ID 的识别结果列于下表：

表 1.3 命名空间 ID 的识别结果

命名空间 ID	识别的命名空间	识别的路径	识别结果
1.8	<code>minecraft</code>	<code>something</code>	<code>minecraft:something</code>
1.9	<code>minecraft</code>	<code>something</code>	<code>minecraft:something</code>
1.10	<code>custom</code>	<code>something</code>	<code>custom:something</code>
1.11	- (含有非法字符 <code>/</code>)	-	识别失败
1.12	<code>minecraft</code>	<code>minecraft/something</code>	<code>minecraft:minecraft/something</code>
1.13	<code>minecraft</code>	- (含有非法字符 <code>S</code>)	识别失败
1.14	- (冒号数量大于 1)	-	识别失败

一般而言，命名空间和路径推荐的写法是**蛇形命名法 (Snake case)**，即当名称中含有多个单字时，以下划线 `_` 取代每一个空格的写法。蛇形命名法的书写仍需遵守合法字符的规定，不能出现大写字母。例如，下面的命名空间 ID 在命名空间和路径上均使用了蛇形命名法：

```
ancient_city:get_out
```

1.15

1.1.4 数据包标签

一个单独的命名空间 ID 只能映射至单独的一个对象，如果要同时映射多个对象，一般的做法是将对象分类，通过映射同一种类别的对象从而映射多个对象。这种将游戏资源分类的手段被称为**数据包标签 (Tags in data packs)**，简称**标签 (Tag)**。由于命令系统存在多个名为“标签”的概念，笔者不建议使用这样的简称以防止与其他概念的混淆。原版游戏有一些既有数据包标签，数据包标签的名称大多拥有实际的意义：例如，数据包标签 `#fire` 映射至两种方块，即 `fire`（火焰）和 `soul_fire`（灵魂火焰）；`#mineable/axe` 映射至所有能被斧采集的方块。

数据包标签的表示方式类似于命名空间 ID，但需要在前面加上井号 `#`，写法为

```
#<namespace>:<id>
```

1.16

例如 `#minecraft:fire`。数据包标签映射的对象可以直接是一个游戏资源，如方块、实体等，也可以是另一个数据包标签。例如，数据包标签 `#minecraft:mineable/axe` 还包含了 `#minecraft:planks`（所有种类的木板）、`#minecraft:signs`（所有种类的告示牌）等数据包标签。指定某数据包标签时，其映射的其他数据包标签下的对象也会被选择。但是同一个数据包标签映射的资源类型必须相同，不能将不同类型的对象放入一个数据包标签中，例如，猪和石头不能被放在同一个数据包标签中。

数据包标签涵盖的对象类型非常广，包括方块、实体、物品、游戏事件、生物群系等。读者可以在既有数据包标签的基础上，使用数据包添加一些自定义的数据包标签。

1.2 命令

命令 (Command)，又称**控制台命令 (Console command)**、**斜杠命令 (Slash command)**或**MC-CMD**，是一种高级的、通过输入具有特定语法文本以实现控制游戏本身运行的功能。命令文本需要讲究严格的语法，不允许任何模糊的表达。目前 MC-CMD 已被正式确认为一种编程语言，名称为 `mcfuction`，与 C 语言、Java、Python 等并列——但这是一种只适用于游戏 Minecraft 内部的编程语言，无法与外部环境进行交互。

1.2.1 命令参数

参数是命令的组成部分，每一条命令都由一个命令头和若干参数组成，参数之间用空格分隔，由此得到命令的通用格式：

```
<命令名> [<参数 1>] [<参数 2>] ...
```

1.17

例如在下面的命令中，`say` 是该命令的命令名，`Hello World!` 是后续参数：

say Hello World!

1.18

在本教程中,为了方便解释命令中各参数的语法,会使用一系列括号或其他符号来表示这些参数的具体内容及可用性。为了使说明更加方便,本教程采用了和游戏本体、Minecraft Wiki 一样的命令语法格式,如下表所示。

表 1.4 语法指引格式

语法	含义	举例
字面量	按字面量原样输入	命令 <code>/locate biome</code> 中第 2 个参数 <code>biome</code> 即为字面量,编写命令时不要更改这个参数的写法。
<参数>	需要使用一合适的值来替换该参数	命令 <code>/say <message></code> 的第 2 个参数需要用自定的值,比如 <code>/say Hello World!</code> 。
[<参数>]	该参数是可选的,如果使用该参数,则需要使用一合适的值替换之	命令 <code>/summon <entity> [<pos>] [<nbt>]</code> 中第 3、4 个参数可选,填写这些参数时需要用自定的值。
(参数 参数)	(必须的)在显示的值中选择一个填写。语法介绍中这些值由竖线分隔	命令 <code>/time query (daytime gametime day)</code> 的第 3 个参数是必填的,从 <code>daytime</code> 、 <code>gametime</code> 和 <code>day</code> 中选择一个填写。
[参数 参数]	(可选的)在显示的值中选择一个填写。语法介绍中这些值由竖线分隔	命令 <code>/experience add <targets> <amount> [levels points]</code> 的第 5 个参数虽然不是必写的,但仍可以从参数 <code>points</code> 和 <code>levels</code> 中选择一个填写。
-> 子命令	必须接入一条子命令	命令 <code>/execute positioned <pos> -> execute</code> 的第 6 个参数是必填的,内容为命令 <code>/execute</code> 的一个子命令。
-> [子命令]	可以接入一条子命令	命令 <code>/execute (if unless) block <pos> <block> -> [execute]</code> 的第 7 个参数可以填写 <code>/execute</code> 的子命令,也可以不填写。

下面举一实例以说明之:

例 1.4

命令 `/data` 的一种语法如下所示:

`data get (block <targetPos>|entity <target>|storage <target>) [<path>]
[<scale>]`

1.19

解

语法中 `data` 为命令名, `get` 是字面量,这两者必须按语法指引中的字面量原样输入命令。第 3、4 个参数必须从 `block <targetPos>`、`entity <target>` 和 `storage <target>` 中选则一种,且不得空缺。若使用 `block <targetPos>`,则 `block` 按原样输入,后续使用 `<targetPos>` 的自定义值,但不得在 `block` 后续使用 `<target>` 参数,因为 `<target>` 是 `entity` 或 `storage` 的后续参数。第 5、6 个参数 `[<path>]`、`[<scale>]` 可选并自定义值。使用该语法且实际可行的命令可以是:

`data get entity @s SelectedItem`

1.20

13

命令 1.20 使用了参数 `entity`，`@s` 是 `<target>` 使用的值，`SelectedItem` 是 `<path>` 使用的值，参数 `<scale>` 未使用。

为了使不同的命令具有不同的功能，它们使用的参数类型各不相同。有些命令作用的对象为实体，它们则会使用指定实体的参数，有些命令作用的对象为某一个坐标，则其使用坐标参数。游戏使用的命令参数有如布尔值、整型、函数、槽位值、坐标值、目标选择器、JSON、NBT 等。一些复杂参数会在本教程后文呈现，下面列举的是基本参数，即在 Brigadier 中使用的六种基本数据类型：

1. 布尔值 (Boolean)

只有两种可用参数，为 `true` 和 `false`，分别代表“是”与“否”。

2. 整型 (Integer)

使用 32 位整型数值，即介于 `-2147483648` 和 `2147483647` 之间的整数值，如 `1`、`0`、`-1` 等。不同命令中使用整型的参数规定的最大可用值和最小可用值不一致。

3. 长整型 (Long)

使用 64 位整型数值，即介于 `-9223372036854775808` 和 `9223372036854775807` 之间的整数值。

4. 单精度浮点数 (Float)

使用占据 4 字节的浮点数，范围大约介于 -3.4×10^{38} 和 3.4×10^{38} 之间，在不同命令中使用单精度浮点数的参数规定的最大可用值和最小可用值不一致。一些单精度浮点数的示例有：`0`、`1.1`、`-1`、`.5` 等，小数形式的整数部分可以省略。在命令参数中使用的浮点数暂时不支持科学计数法^①。

5. 双精度浮点数 (Double)

使用占据 8 字节的浮点数，范围大约介于 -1.8×10^{108} 和 1.8×10^{108} 之间。可以表示比单精度浮点数绝对值更大的有效数字。

6. 字符串 (String)

(1) 单个词 (Single word)

即不含空格的字符串，如 `word`，若单个词的内容由多个词语组成，则一般使用下划线 `_` 连接相邻词，如 `word_with_underscores`。

(2) 词组 (Quotable phrase)

可以由双引号括起，如 `"quoted phrase"`，也可以使用单引号来定义，如 `'quoted phrase'`，此时单词之间可以有空格。

(3) 贪婪词组 (Greedy phrase)

这种形式的词组不带引号，任意使用空格。该形式的参数通常位于命令的末尾，将命令的剩余部分全部作为字符串参数。如：`words with spaces`。上文中命令 1.18 就使用了这种参数。

1.2.2 命令的输入

命令是一种文本输入，以下是可供命令输入的途径：

1. 使用聊天栏输入命令

^① 参见 MC-130925。

为了和普通的聊天文本区分开来,在聊天栏中输入命令时会在命令前加一个前缀 `/`,此前缀必不可少。在不使用按键 `T` 召唤聊天栏时可以直接键入 `/` 输入命令,这是使玩家快速进入命令输入模式的一种办法。

呼出聊天栏后,可以使用 `↑` 或 `↓` 键调用**命令历史 (Command history)**,即先前键入的命令。如果之前输入的命令有语法错误的话,切换至该命令时依旧会有语法错误,不会自动更正,更不会因为含有语法错误就不显示该命令。这种快捷键在命令方块控制台中不适用。命令历史可以跨存档调用。

在聊天栏输入命令时, `Tab` 键可用于补全命令。未输入任何命令字符的时候,使用 `Tab` 键可以看到聊天栏上出现的一个命令列表 (如图 1.1),鼠标滚轮有助于翻找需要的命令。

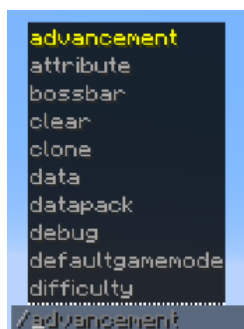


图 1.1 在聊天栏输入命令时使用 `Tab` 键

读者可以直接在命令列表中点击需要的命令,或者如图 1.2 (a) 所示,输入命令的前若干字符后使用 `Tab` 键补全。若这个命令后续还有其他参数,则也可以如图 1.2 (b) 所示用 `Tab` 键补全。



图 1.2 命令输入过程

聊天栏的字符数量被限制在 256 以内,命令开头的 `/` 也会被计入字符数。因此,聊天栏不能用于执行太长的命令。

2. 在命令方块或命令方块矿车内输入命令

命令方块控制台可输入的字符最多为 32500 个,较聊天栏的限制有很大提升。文本框长度有限,每次只能显示命令的其中一段,需要使用 `鼠标左键` 或 `←`、`→` 键移动光标调整命令显示的位置,且每次打开命令方块 GUI 时光标总显示在命令的末尾。

在命令方块或命令方块矿车内输入命令时,斜杠前缀 `/` 不是必须的。和在聊天栏中使用命令一样,当文本框中无任何内容时,下方会显示一个命令列表 (如图 1.3),通过调整鼠标滚轮能够调整命令列表显示的位置,按 `Tab` 键能够在输入命令时自动补全或选择命令的部分。



图 1.3 命令方块 GUI

3. 在数据包函数文件中输入命令

这种编写方式需要使用一定的编译软件，常用的编译软件有 Windows 自带的记事本、Visual Studio Code 等。函数中的命令不能带有斜杠前缀。具体的内容可参阅《数据包》教程的描述。

4. 在服务器控制台中输入命令

5. 在带有 `run_command` 动作的点击事件的文本组件或对话框按钮中输入命令

1.2.3 权限等级与限制条件

命令功能强大、种类繁多，如果在任意情况下都能够随意使用，则很有可能会破坏玩家的游戏体验。因此，命令系统有一套专门的机制用于控制游戏内可用命令的情形，即权限等级。**权限等级 (Permission level)** 用于决定命令执行者可以使用什么样的命令。所有命令都有一个所需的权限等级，如果命令执行者没有达到该有的权限等级，则无法执行该命令。例如：`/advancement` 需要的权限等级为 2，命令方块的权限等级也为 2，因此命令方块可以执行该命令；而关闭命令的单人游戏玩家的权限为 0，所以该玩家不能执行该命令。

权限等级共分为 0、1、2、3、4 级，表罗列了 Java 版所有可用命令需要的权限等级与限制条件。除权限等级之外，一些命令还对当前的游戏世界有限制：一些命令只能在专用服务器（以下简称多人游戏）中使用，另有只能在非专用服务器（以下简称单人游戏，无论是否对局域网开放）中使用的命令，然而大部分命令都是无此限制条件的。

表 1.5 Java 版可用命令列表

命令	权限等级	限制条件	命令	权限等级	限制条件
<code>/advancement</code>	2		<code>/attribute</code>	2	
<code>/ban</code>	3	仅多人游戏	<code>/ban-ip</code>	3	仅多人游戏
<code>/banlist</code>	3	仅多人游戏	<code>/bossbar</code>	2	
<code>/chase</code>	0	仅调试工具	<code>/clear</code>	2	
<code>/clone</code>	2		<code>/damage</code>	2	
<code>/data</code>	2		<code>/datapack</code>	2	
<code>/debug</code>	3		<code>/debugconfig</code>	3	仅调试工具
<code>/debugmobspawning</code>	2	仅调试工具	<code>/debugpath</code>	2	仅调试工具
<code>/defaultgamemode</code>	2		<code>/deop</code>	3	仅多人游戏

(续表)

命令	权限等级	限制条件	命令	权限等级	限制条件
<code>/dialog</code>	2		<code>/difficulty</code>	2	
<code>/effect</code>	2		<code>/enchant</code>	2	
<code>/execute</code>	2		<code>/experience</code>	2	
<code>/fill</code>	2		<code>/fillbiome</code>	2	
<code>/fetchprofile</code>	2		<code>/forceload</code>	2	
<code>/function</code>	2		<code>/gamemode</code>	2	
<code>/gamerule</code>	2		<code>/give</code>	2	
<code>/help</code>	0		<code>/item</code>	2	
<code>/jfr</code>	4		<code>/kick</code>	3	
<code>/kill</code>	2		<code>/list</code>	0	
<code>/locate</code>	2		<code>/loot</code>	2	
<code>/me</code>	0		<code>/msg</code>	0	
<code>/op</code>	3	仅多人游戏	<code>/pardon</code>	3	仅多人游戏
<code>/pardon-ip</code>	3	仅多人游戏	<code>/particle</code>	2	
<code>/perf</code>	4		<code>/place</code>	2	
<code>/playsound</code>	2		<code>/publish</code>	4	仅单人游戏
<code>/raid</code>	3	仅调试工具	<code>/random</code>	0 (不使用 <code>sequence</code>) 或 2 (使用 <code>sequence</code>)	
<code>/recipe</code>	2		<code>/reload</code>	2	
<code>/return</code>	2		<code>/ride</code>	2	
<code>/rotate</code>	2		<code>/save-all</code>	4	仅多人游戏
<code>/save-off</code>	4	仅多人游戏	<code>/save-on</code>	4	仅多人游戏
<code>/say</code>	2		<code>/serverpack</code>	2	仅调试工具
<code>/schedule</code>	2		<code>/scoreboard</code>	2	
<code>/seed</code>	0(单人游戏)或2(多人游戏)		<code>/setblock</code>	2	
<code>/setidletimeout</code>	3	仅多人游戏	<code>/setworldspawn</code>	2	
<code>/spawn_armor_trims</code>	2	仅调试工具	<code>/spawnpoint</code>	2	
<code>/spectate</code>	2		<code>/spreadplayers</code>	2	
<code>/stop</code>	4	仅多人游戏	<code>/stopsound</code>	2	
<code>/stopwatch</code>	2		<code>/summon</code>	2	
<code>/swing</code>	2		<code>/tag</code>	2	
<code>/team</code>	2		<code>/teammsg</code>	0	

(续表)

命令	权限等级	限制条件	命令	权限等级	限制条件
<code>/teleport</code>	2		<code>/tell</code>	0	
<code>/tellraw</code>	2		<code>/test</code>	2	
<code>/tick</code>	3		<code>/time</code>	2	
<code>/title</code>	2		<code>/tm</code>	0	
<code>/tp</code>	2		<code>/transfer</code>	3	仅多人游戏
<code>/version</code>	0		<code>/version</code>	0(单人游戏) 或 2(多人游戏)	
<code>/w</code>	0		<code>/waypoint</code>	2	
<code>/warden_spawn_tracker</code>	2	仅调试工具	<code>/weather</code>	2	
<code>/whitelist</code>	3	仅多人游戏	<code>/worldborder</code>	2	
<code>/xp</code>	2				

1.2.4 命令的解析 *

游戏处理命令的过程可分为**解析**和**执行**两个阶段。Minecraft 使用 **Brigadier** 作为命令的解析器、派发器。

命令的实质是一个根命令节点的直接量分支,这就意味着所有的命令都是一个树状结构,命令中每一个参数都作为一个节点,而命令名作为根节点使用。显然,一个节点可能会有两种类型:字面量和变量,反映到命令文本语法中分别为字面量参数和需要自定义值的参数。

游戏在读取命令后,会首先解析根节点是否是已注册的命令,其次解析下一个参数即子节点是否可用,然后依次解析余下的节点。**Brigadier** 读取到某一个节点时,会枚举其子节点的所有可行节点,并在聊天栏或命令方块控制台内显示为可读性较强的可视化参数列表。

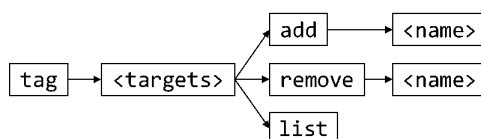


图 1.4 命令 `tag` 的所有结点和分支

以命令 `/tag` 为例,其命令树如图 1.4 所示。`tag` 是根命令,其子节点 `<target>` 是一个需要特定参数类型(这里是 `entity`)的节点,解析此节点的时候,会判断输入的参数是否为 `entity` 类型,若为否则解析异常,命令无法执行。2 级子节点是已注册的字面量 `add`、`remove` 和 `list`,解析该级节点的工作比较简单:只需读取该节点的文本是否与注册的字面量吻合。若 2 级子节点的参数指定为 `add`、`remove`,则读取 3 级子节点 `<name>`,这个节点又是一个需要自定义的量;若 2 级子节点的参数指定为 `list`,则不能再添加后续参数。

1.2.5 命令上下文

当一条命令被执行时,该命令一定有一个调用者以及调用环境,这一系列调用者及调用环境构成的集合被称为**命令上下文 (Command context)**,或称**执行上下文 (Execution context)**、**命令源 (Command origin)**、**命令来源堆叠 (Command source stack)**。

命令上下文由以下参数构成:

1. 执行权限等级

2. **执行者 (Executor)**

由“执行者名称”和“执行者实体”两个参数构成,但执行者实体不一定存在,例如执行者为命令方块、命令方块矿车或服务端的时候。

3. **执行位置 (Execution position)**

这个参数是命令执行时所在的坐标,包含 x 、 y 、 z 三个坐标参数。

4. **执行朝向 (Execution rotation)**

这个参数是命令执行时面向的方向,包含偏航角和俯仰角两个参数。

5. **执行锚点 (Execution anchor)**

这个参数是局部坐标的原点,当执行者为实体时,这个参数可以指定执行的锚点基于实体的脚部还是眼部,因此有脚部和眼部两个可用参数。其中脚部即为原本的执行位置,眼部为原本的执行位置在 y 轴方向加上实体眼睛的高度。

6. **执行维度 (Execution dimension)**

这个参数是命令执行所在的维度,执行位置位于这个维度内。

7. 执行输出反馈

尝试执行命令会产生一定的执行效果,并在执行失败或执行成功时返回**成功次数 (Success)**和**结果 (Result)**两个返回值。其中成功次数总是为 0 或 1,结果一定为整数,遇到小数时则向下取整。下面讨论所有种类的命令执行效果:

(1) **无法解析**

这种效果会在命令中存在无法解析的参数、输入的命令不完整或执行上下文不符合命令的限制条件,如执行者拥有的权限等级不够、超出游戏世界的限制时出现。此时命令没有返回值。在聊天栏或命令方块内输入的命令若无法解析,则会返回语法错误信息。在函数内的命令若无法解析,则该函数无法加载。

(2) **执行错误**

出现这种效果说明命令中存在严重的漏洞。此时命令没有返回值。

(3) **Void**

当且仅当执行命令 `/function` 时会出现这种效果,说明 `/function` 调用了一个 void 类型的函数,没有返回值。

(4) **执行中断**

当且仅当执行命令 `/execute` 时会出现这种效果,此时 `/execute` 的分支数量为 0,在 `run` 子命令执行前执行就已经中止。此时命令没有返回值。

(5) **执行失败和 执行成功**

当命令执行效果不是上述 4 种中任意一种时,才能出现执行失败或执行成功。且只有当执行效果为执行失败或执行成功时,才能返回成功次数和结果。执行失败并不意味着命令没有起作用,执行成功也不意味着命令会对游戏做出更改。

不同情况下的命令上下文列举于下表:

表 1.6 不同情况的命令上下文

情况	执行权限等级	执行者	执行位置	执行朝向	执行锚点	执行维度
玩家	视情况而定	玩家	玩家的位置	玩家的朝向	脚部	玩家所在的维度
服务器控制台	4	Server	世界出生点 方块的西北 下角顶点	水平向南	脚部	主世界
服务端	2	Server	世界出生点 方块的西北 下角顶点	水平向南	脚部	主世界
命令方块	2	命令方块	命令方块的 正中心	命令方块的 朝向	脚部	命令方块所 在维度
命令方块矿 车	2	命令方块矿 车	命令方块矿 车的位置	命令方块矿 车的朝向	脚部	命令方块矿 车所在维度
函数	可修改	该函数的调 用者	函数调用者 的位置	函数调用者 的朝向	函数调用者 的锚点	函数调用者 所在维度
告示牌	2	点击告示牌 的玩家	告示牌所在 方块正中心	水平向南	脚部	告示牌所在 维度
<code>/execute</code>	-	修饰后的执 行者	修饰后的执 行位置	修饰后的执 行朝向	修饰后的执 行锚点	修饰后的执 行维度
自定义进度	2	获得进度的 玩家	玩家的位置	玩家的朝向	脚部	玩家所在的 维度
自定义魔咒	2	魔咒作用的 实体	魔咒作用的 位置	魔咒作用实 体的朝向	脚部	魔咒作用的 维度

小提示

玩家的权限等级与其游戏模式无关，需要分情况讨论：

1. 若该玩家是服务器管理员，则他的权限等级由`ops.json`中的值决定，默认为 4 级；
2. 若该玩家处于启用命令的单人世界中或为启用命令的局域网世界所有者，则他的权限等级为 4 级；
3. 若该玩家处于启用命令的局域网世界中，则他的权限等级为 4 级；
4. 非上述情况者权限等级一律为 0 级。

函数的权限等级默认为 2 级，可在`server.properties`中修改。

1.3 JSON 格式

在讲述数据包和资源包这两种开发实例之前，有必要先介绍 Minecraft 使用的用于数据交换的格式，即 JSON 格式。事实上，这种数据格式的应用极其广泛，是当今互联网最通用的数据交

换格式之一。在 Minecraft 中，为方便网络传输和数据存储，一些游戏内程序会被**序列化**为 JSON 格式。而对于数据包、资源包内容中的 JSON 格式而言，游戏会尝试将这些 JSON 数据**反序列化**为游戏内相应的程序。

游戏的一些数据以 `.json` 文件的格式存储在各文件夹中。这些文件有如：资源包的模型文件，数据包的进度、谓词文件，游戏版本信息文件，等等。下面展示的是金合欢木按钮的模型文件：

```
assets\minecraft\models\block\acacia_button.json
1 {
2   "parent": "minecraft:block/button",
3   "textures": {
4     "texture": "minecraft:block/acacia_planks"
5   }
6 }
```

1.3.1 JSON 数据类型

JSON (JavaScript Object Notation, JavaScript 对象表示法) 是一种轻量级数据交换格式，独立于编程语言，是 JavaScript 的一个子集。其内容主要由键和值构成，即**键值对 (Name-value pair)**，这些键值对可认为是一个个**字段 (Field)**。这种格式主要有两个优点：第一，便于编写者阅读和修改；第二，由于其轻量级的特点，其对环境的依赖程度较小，因此能用于存储大量不同种类的信息。Minecraft 使用的 JSON 标准为 ECMA-404。

JSON 格式键值对的基本语法为：

```
"<键>":<值>
```

1.21

对于一个键，可以给它定义一个值。在书写时，JSON 的所有键的键名必须用**双引号**引起。若有多个键值对，则需使用逗号将这些键值对分隔开来，最后一个键值对的后面不加逗号，如：

小提示

键名的两侧必须是英文引号，且不接受单引号！

```
"<键>":<值>,"<键>":<值>
```

1.22

在一个 `.json` 文件中，须使用花括号 `{}` 将所有的键值对封装包裹在一起，如：

```
{"<键>":<值>,"<键>":<值>}
```

1.23

对于值而言，每一个不同的键都需的值的类型不尽相同，比如键 `color` 可能需要的是颜色值，`bold` 可能需要的是布尔值，`text` 可能需要的是字符串，等等。JSON 一共使用六种不同的数据类型：

1. **字符串 (String)**

常见的数据类型，可以包含任意字符（如空格），字符串由一对**（英文）双引号**定义，**不接受单引号**，用法举例：

```
"description": "The default data for Minecraft"
```

1.24

也可以使用中文：

```
"description": "我的世界默认数据包"
```

1.25

JSON 同时也支持 Unicode，表示方式为 `\uxxxx`，其中每一个 `x` 都为十六进制数字。例如，符号★的 Unicode 为 `U2605`，则在字符串中输入★的方式可以为：

```
"text": "\u2605"
```

1.26

这样便可以在字符串中输入一些生僻字或是在键盘上无法直接打出来的字符。但是 Minecraft 的字库是有限的，并非所有的字符都可以在 Minecraft 中显示。

2. 布尔值 (Boolean)

由 `true`（真）或 `false`（假）定义，这两者是 JSON 中的字面量符号，不需要使用双引号引起，举例：

```
"bold": true
```

1.27

```
"italic": false
```

1.28

3. 数值 (Number)

由数字定义，允许使用整数、浮点数或是科学计数法表示的数，举例：

```
"min": 1.0
```

1.29

在 JSON 中使用的数值不需要注明它们的数据类型。

4. 数组 (Array，或称为列表)

由一对方括号定义，数组中元素与元素之间使用逗号隔开，**最后一个元素后不能有逗号**。这些元素可以是其他的数据类型，如字符串、布尔值、数值和对象，数组中甚至能嵌套数组。在定义其他的数据类型时，需注意这些数据类型的定义方法。以下为包含了数值的数组：

```
"frames": [1, 2, 3, 4, 5]
```

1.30

下面为包含了字符串的数组，字符串均由一对双引号定义：

```
"text": ["A", "B", "C"]
```

1.31

对于数组内的元素，其数据类型不必完全一致，例如：

```
"extra": [1, {"text": "2"}, "3"]
```

1.32

5. 对象 (Object)

由一对花括号定义，对象内字段与字段之间使用逗号隔开，**最后一个字段后不能有逗号**。对象中可以包含其他数据类型，也可以在对象中嵌套对象。整个 `.json` 文件就可以看作是一个大的对象。在编写 JSON 的时候，通常需要用对象嵌套对象，因此花括号一定要检查是否匹配。用法举例：

```
{
  "rolls": {
    "type": "minecraft:binomial",
    "n": 3,
    "p": 0.2
  }
}
```

1.33

6. Null

空值，作为字面量符号使用。Minecraft 基本不使用这种数据类型。

由于 JSON 为多层级结构，为了方便说明，本系列教程会使用 and Minecraft Wiki 一样的树状图来表示。例如：



对应的 JSON 为：

```
{
  "string": "这是一个字符串",
  "boolean": true,
  "number": 5,
  "array": [
    "这是数组的第一个元素，是一个字符串",
    {
      "string": "这是对象内的一个字符串"
    }
  ],
  {
    "string": "这是对象内的一个字符串"
  }
}
```

1.34

小提示

如何看懂树状图？

1.3.2 JSON 的转义序列

使用 JSON 字符串时，如果字符串本身的内容中含有英文引号 `"`，如一个 JSON 字段 `text` 的值需要为 `"Hello World!"`，那该如何编写 JSON 呢？若使用如下的 JSON：

```
"text":"Hello World!"
```

1.35

这样通常会产生报错，这是由于用于定义字符串的引号和值中的英文引号发生了配对从而导致了错误，因此需要使用**转义字符 (Escape character)** `\` 对文本引号进行转义。转义的作用为：将被转义的字符转换成字符，被转换的引号便不再与用于定义字符串的引号发生配对。除用于转义英文引号外，反斜杠还可以用于转义反斜杠以及创造一些特定的转义序列。JSON 中可用的转义序列如下：

1. `\"`，是半角双引号（英文引号）`"` 的转义方式（中文引号不需要转义）；
2. `\\`，是反斜杠 `\` 的转义方式，已被转义的反斜杠被视为普通字符，不再具有转义作用；
3. `\b`，退格；
4. `\f`，换页；
5. `\n`，换行；
6. `\r`，回车；
7. `\t`，制表符；
8. `\u<unicode>`，用四位十六进制表示 Unicode 字符。

小提示

`\b`、`\f`、`\n`、`\r`、`\t` 这些特殊的转义序列能在 JSON 中使用，但这不代表这些转义序列能在相应的游戏实例中真正起作用。例如，在物品修饰器中定义物品名称时，虽然 JSON 支持输入换行符 `\n`，但物品名称本身不支持换行。

由此可以得出字段 1.35 的正确写法：

```
"text":"\\Hello World!\\"
```

1.36

同样地，如果值为 `\\Hello World!\\`，需要对反斜杠进行转义，正确的写法为：

```
"text":"\\\\Hello World!\\\\"
```

1.37

例 1.5

判断下列 JSON 的转义是否有效，若有效，则写出其值。

```
"text":"\\Hello World\\"
```

1.38

```
"text":"\\Hello World!\\\\"
```

1.39

```
"text":"\\Hello World!\\\\"
```

1.40

```
"text":"\\Hello World!\\\\"
```

1.41

解

字段 1.38 的值出现了连续三个反斜杠 `\\` 的情况，第一个反斜杠用于转义第二个反斜杠，而第二个反斜杠不再具有转义作用；而第三个反斜杠后面没有其他转义序列，故值无效。

对于 1.39, 依次检验所有反斜杠: 第一个反斜杠用于转义引号, 第二个反斜杠用于转义第三个反斜杠, 第四个反斜杠用于转义第五个反斜杠, 第六个反斜杠用于转义引号。故值为 `"\\Hello World\\\"`。

字符串两端的反斜杠数量不一定需要相等, 因此字段 1.40 是有效的, 输出结果 `"Hello World\"`。

注意, 1.41 的所有转义序列都书写正确, 但第三个反斜杠后面存在一个引号, 而第三个反斜杠已被第二个反斜杠转义而失去转义作用。因此用于定义字符串的引号配对混乱, 该值无效。判断一个值是否有效, 不仅需要有效的转义序列, 还要注意字符串本身的双引号是否正确配对。

1.3.3 多级转义 *

特殊情况下, JSON 字段可能会以字符串类型包含另一个 JSON 数据而非直接嵌套为相应类型的值, 例如:

❗ content: `{"text": "Hello World!"}`

字段 **❗ content** 的值类型为字符串, 两端一定需要一对引号。为了防止决定字符串的引号与值中的引号发生匹配混乱, 会将值中的引号进行转义:

```
"content": "{\\"text\\":\\"Hello World!\\"}"
```

1.42

如果 **❗ text** 字段的值也带有引号, 如 `"Hello World!"`, 则需要相应地添加反斜杠:

❗ content: `{"text": "\\\"Hello World!\\"}"`

1.4 文件结构

1.5 数据包

Minecraft 的命令系统虽然完善, 但其功能十分有限。例如, 命令没有办法直接指导游戏世界的生成; 直接用命令模拟一些游戏机制也不够灵活。数据包可以看作是命令系统功能的延伸: 它不仅为命令提供了程序化执行的环境, 更开放了部分 API 以允许数据驱动内容。

数据包 (Data pack) 允许玩家在不修改游戏代码的前提下覆盖既有的或添加自定义的游戏内容。数据包本质上是一个文件夹或压缩文件。一个数据包仅对特定的存档有效, 它被储存在 `.minecraft\saves<存档名称>\datapacks` 中。数据包可以是文件夹, 也可以是 `.zip` 类型的压缩文件。同一个 **datapacks** 文件夹内能存放多个数据包。

数据包有两种添加方式——

1. 手动添加：直接将数据包添加至 `.minecraft\saves\<存档名称>\datapacks`。
2. 创建世界时添加数据包：在创建新的世界界面，选择 **更多**，点击 **数据包** 选项，此时会进入选择数据包窗口，类似于资源包选项的窗口，可在“可用”一栏内选用数据包，只有“已选”一栏的数据包有效，且数据包的加载顺序可以在该栏中调换。点击 **打开包文件夹** 选项后游戏会弹出一个临时的文件夹，此时可以将数据包拖入其中。



图 1.5 选择数据包窗口

当一个存档中存在多个有效的已启用数据包时，游戏会根据数据包的顺序加载其内容，这里的“有效”是指数据包有合法的元数据且数据包内无任何语法错误。已启用数据包的加载顺序存储于 `level.dat` 中。在选择数据包窗口“已选”一栏的加载顺序表现为从下到上。

若这些数据包对同种资源进行定义，则**后加载的数据包会对先加载的数据包进行覆盖**，表明越靠后加载的数据包其优先级越高。可使用命令 `/datapack` 查询、修改、控制这些数据包的启用或禁用，`/datapack` 所需的权限等级为 2，以下是所有用法：

1.6 资源包

1.7 游戏机制

游戏为命令提供了一个运行环境，为此命令系统不免受到游戏机制的制约。在时间上，命令受到游戏循环驱动的影响，以游戏刻为单位执行；在空间上，命令受到区块加载的影响，只能在允许运算的区块中执行。本节旨在介绍游戏加载、运行、更新的一些基本游戏机制。

1.7.1 端

Minecraft 的架构是**客户端-服务端模型**，顾名思义，Minecraft 使用**客户端（Client）**和**服务端（Server）**来运作自身。这两个**端（Sides）**之间的通信是由**封包（Packet）**实现的。在网络工程中，这个概念一般译为“数据包”，而 Minecraft 中另有一个叫 Datapack（数据包）的概念，故 Packet 在 Minecraft 技术性开发领域会特地译为“封包”。

然而，仅通过客户端和服务端理解 Minecraft 的运作是远远不够的，因为 Minecraft 的架构还包括**物理端（Physical sides）**和**逻辑端（Logical sides）**，并且物理端和逻辑端分别具有各自的客户端和服务端。

一、物理客户端

物理客户端（Physical client）是指下载游戏版本得到的 `<version>.jar` 文件，它的默认文件路径为 `.minecraft\<版本号>\<version>.jar`。物理客户端包含了游戏的全部内容，也包含了内置的客户端和服务端，即**逻辑客户端（Logical client）**和**逻辑服务端（Logical server）**，其中逻辑服务端又称**内置服务器（Integrated server，或译为集成服务端）**。内置服务器会受到客户端的影响。

逻辑客户端负责接收来自玩家的输入、处理资源包、渲染游戏画面，并将数据输送给逻辑服务端处理；逻辑服务端负责处理由客户端发送的数据，运行游戏逻辑。例如，当玩家在游戏中移动时，客户端会根据玩家输入的移动方向渲染玩家此时的游戏画面，同时又将玩家移动的信息通过封包发送给逻辑服务端，逻辑服务端计算玩家的坐标、玩家周围是否存在任何的碰撞箱阻止玩家移动，将计算结果通过封包返还给逻辑客户端，渲染玩家移动的游戏画面。客户端的渲染会与服务端产生不一致的情况，例如标记是一种仅存在于服务端的实体，在客户端上并不会渲染标记，参见小节 3.1。

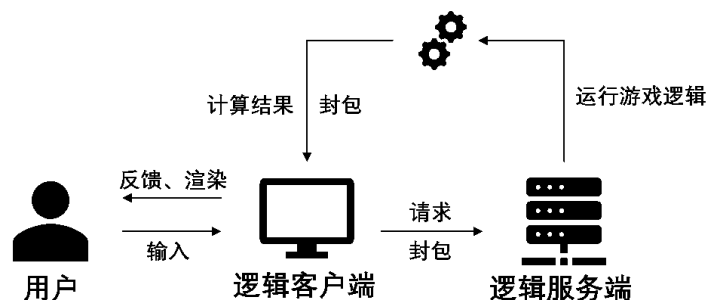


图 1.6 逻辑客户端和逻辑服务端的运行流程

即使是进行单人游戏，Minecraft 依旧会在玩家进入本地世界时创建一个**内置服务器**，在本地世界关闭时**内置服务器即被关闭**。这个内置服务器可以开放至局域网，从而将单人游戏开放为局域网联机的多人游戏。此时内置服务器拥有一个地址，其格式为

`<IPv4 地址>:<端口>`

1.43

局域网联机的 IPv4 地址可由 CMD 的 `ipconfig` 命令查询。端口是一个数值，可以自由指定，范围为 `0` 至 `65535`（含两端）。除了通过暂停游戏的对局域网开放选项外，玩家还可以通过命令 `/publish` 开放内置服务器，该命令所需权限等级为 4，且仅能在单人游戏中使用，其语法为：

`publish [<allowCommands>] [<gamemode>] [<port>]`

1.44

其中的参数^①：

1.8 服务器

1.9 带有简单参数的命令指引

① 括号中内容为该参数的类型及该参数类型在注册表内的命名空间 ID，后续教程均如此。

第二章 坐标

Minecraft 的游戏世界是三维的。在编写数据包的时候，有时需要确定实例所需的位置参数。这样的参数被称为**坐标 (Coordinate)**。本章将详细介绍各种坐标参数以及这些参数在命令上的应用。

2.1 坐标系与坐标

Minecraft 使用的空间直角坐标系是右手坐标系。在这种空间直角坐标系中， x 轴和 z 轴所反映的是水平方向上的位置， y 轴所反映的是垂直方向上的位置。其中， x 轴的正方向指向正东，而 z 轴的正方向指向正南。

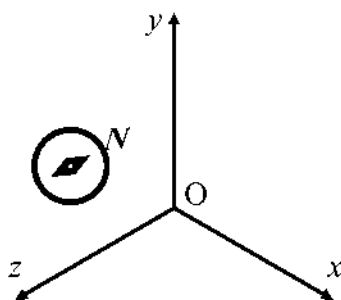


图 2.1 适用于 Minecraft 的空间直角坐标系

在命令参数中，可以用三个分量来表示某一点的位置，这是一个有序的实数三元组：

<x> <y> <z>

2.1

比如，用数学方法表示的点 $(0, 4, 4)$ 在一些命令参数中直接表示为 `0 4 4`。

例 2.1

空间直角坐标系中有两个点 $A(124.5, 76, -64.29)$ 、 $B(-10.003, 80, -33.33)$ ，则 B 点在水平位置上位于 A 点的什么方向？

解

B 点 x 坐标位于 A 点 x 坐标的负方向，因此 B 点在东西方向上位于 A 点的西方； B 点 z 坐标位于 A 点 z 坐标的正方向，因此 B 点在东西方向上位于 A 点的南方。综上所述， B 点位于 A 点的西南方向。

虽然不同的坐标表示方式基本一致，但不同的命令作用的对象不同，其坐标参数类型也不完全一致，下文将梳理命令系统使用的所有种类的坐标参数。

2.1.1 坐标的命令参数类型

命令使用 4 种与坐标相关的参数类型：方块坐标 `minecraft:block_pos`、三维坐标 `minecraft:vec3`、平面方块坐标 `minecraft:column_pos` 和二维坐标 `minecraft:vec2`。它们的关系和应用场景可以很清晰地列于下表：

表 2.1 Minecraft 中的坐标参数

	一般应用于方块	一般应用于非方块的游戏内容
三维	<code>minecraft:block_pos</code>	<code>minecraft:vec3</code>

(续表)

	一般应用于方块	一般应用于非方块的游戏内容
二维	<code>minecraft:column_pos</code>	<code>minecraft:vec2</code>

一、方块坐标

坐标表示的是一个没有体积的点，而方块是有体积的，因此需要给方块规定一个基准点，用这个基准点的坐标来表示该对象的坐标。

Minecraft 规定：大部分方块的长、宽和高均为 1 米，体积为 1 立方米。用坐标系表示这些位置时，默认了方块的边长和坐标系的单位长度在数值上相等，这意味着坐标系的基本单位为米，或称为“格”。

一个方块使用其**西北下角**的点作为它的**方块坐标 (Block position)**。若一个方块的西北下角顶点坐标为 (x, y, z) ，则该方块的方块坐标记为 (x, y, z) ，而这个方块位于 (x, y, z) 和 $(x + 1, y + 1, z + 1)$ 这两个坐标围成的立体几何图形之间。59

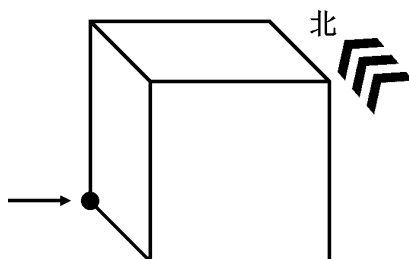


图 2.2 用方块这个方向的顶点来表示方块坐标

由于方块的角总是位于整数坐标点，作为命令参数 `minecraft:block_pos` 的方块坐标一定是由三个整数构成的有序三元组。

例 2.2

如图所示，方块坐标为 `0 0 0` 的方块为哪一个？

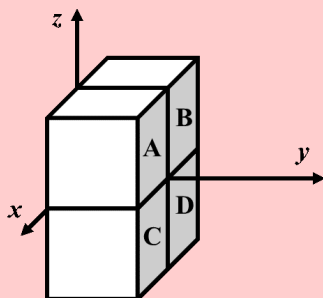


图 2.3

解

方块坐标严格按照西北下角顶点来计算，而正西、正北分别是 x 轴和 z 轴的负方向，因此用方块坐标指示的方块位置，永远位于实际坐标的东南方向，且在垂直方向上位于上方，在空间直角坐标系中的反映即为 x 、 y 、 z 三个坐标轴的正方向。因此方块坐标为 `0 0 0` 的方块位于第一卦限，即方块 A。

二、三维坐标

三维坐标 (Three-dimensional coordinates) 是精确表示一个位置的坐标参数, 命令参数类型为 `minecraft:vec3`, 用于表示坐标位置的三个元素均为双精度浮点数。三维坐标一般应用于实体, 它也可能在粒子生成和声音播放的时候被使用。例如, 这是一个合法的三维坐标:

```
5.0 56.0 17.0
```

2.2

这个坐标带有小数点, 因为三维坐标的三个参数均是双精度浮点数。但是, 这并不意味着三维坐标只能使用浮点数。也可以在三维坐标中使用整数形式, 如:

```
5 56 17
```

2.3

注意, 上述这两个坐标描述的位置并不是一致的。在实际操作中, 却发现这个玩家位于三维坐标 $(5.5, 56.0, 17.5)$ 。如图, 可以观察到玩家的坐标发生了“偏移”, 与实际坐标有所出入。其中 x 坐标和 z 坐标都发生了“偏移”, 而 y 坐标不受影响。

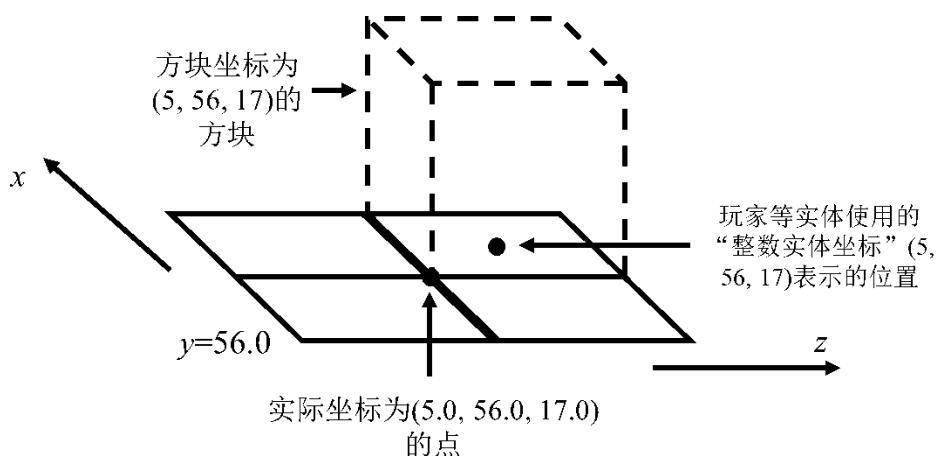


图 2.4 整数坐标发生的“偏移”

这些位置的偏移都位于相对方块两条对边的中心线上, 这是因为三维坐标使用了 **中心校准 (Center correct)**, 即使用整数形式的三维坐标, 当其某一个坐标参数为 n ($n \in \mathbb{Z}$) 时, 其实际坐标为 $n - 0.5$, 这样可以使得实体位置与方块位置相适应。注意**中心校准仅适用于 x 坐标和 z 坐标**。 **y 坐标严格使用实际坐标**。

注意这里不使用“三维坐标根据方块坐标位于方块中心”的说法, 是因为三维坐标的三个参数中整数和浮点数形式可以混用, 并且使用小数形式的参数严格遵循实际坐标, 整数形式的参数则使用中心校准。比如, 位于 `5 56 17.0` 的玩家实际位于 $(5.5, 56, 17.0)$ 。

三、平面方块坐标

故名思义, 平面方块坐标 `minecraft:column_pos` 就是二维的方块坐标, 以西北角的二维坐标作为一个方块纵列的平面坐标, 两个元素均为整数。

四、二维坐标

即只由 x 坐标和 z 坐标构成的 **二维坐标 (Two-dimensional coordinates)**。二维坐标的命令参数类型为 `minecraft:vec2`, 两个元素均为双精度浮点数。二维坐标若为整数, 则也使用中心校准。

第三章 区块格式

3.1 技术性实体

索引

B

扁平化 (The flattening) 8
标签 (Tag) 12
布尔值 (Boolean) 14, 22

C

长整型 (Long) 14
成功次数 (Success) 19
词组 (Quotable phrase) 14

D

单个词 (Single word) 14
单精度浮点数 (Float) 14
端 (Sides) 27
对象 (Object) 22

E

二维坐标 (Three-dimensional coordinates) 32

F

方块坐标 (Block position) 31
封包 (Packet) 27
服务 (器) 端 (Server) 27

G

固有注册表 (Built-in registry) 2

J

键值对 (Name-value pair)	21
结果 (Result)	19
JSON (JavaScript Object Notation, JavaScript 对象表示法)	21

N

内置服务器 (Integrated server)	27
---------------------------	----

Q**K**

权限等级 (Permission level)	16
-------------------------	----

客户端 (Client)	27
可写注册表 (Writable registry)	2
控制台命令 (Console command)	12

S**L**

逻辑端 (Logical sides)	27
逻辑服务端 (Logical server)	27
逻辑客户端 (Logical client)	27

三维坐标 (Three-dimensional coordinates)	32
蛇形命名法 (Snake case)	11
双精度浮点数 (Double)	14
数据包 (Data pack)	25
数据包标签 (Tags in data packs)	12
数值 (Number)	22
数组 (Array)	22

M

MC-CMD	12
命令 (Command)	12
命令来源堆叠 (Command source stack)	19
命令历史 (Command history)	15
命令上下文 (Command context)	19
命令源 (Command origin)	19
(赋)命名空间 ID (Namespaced identifier)	9, 10
命名空间字符串 (Namespaced string)	9

T

贪婪词组 (Greedy phrase)	14
----------------------	----

W

物理端 (Physical sides)	27
物理客户端 (Physical client)	27

X

斜杠命令 (Slash command) 12

Z

整型 (Integer) 14
执行朝向 (Execution rotation) 19
执行锚点 (Execution anchor) 19
执行上下文 (Execution context) 19
执行维度 (Execution dimension) 19
执行位置 (Execution position) 19
执行者 (Executor) 19
中心校准 (Center correct) 32
转义字符 (Escape character) 24
注册表 (Registry) 2
字段 (Field) 21
字符串 (String) 14, 21
资源标识符 (Resource identifier) 9
资源路径 (Resource location) 9
坐标 (Coordinate) 29